

Computer Science & IT

Data Structure & programming



Linked List

Lecture No. 04



By- Abhishek Sir

Recap of Previous Lecture



Topic

operation on Doubly linked list

Topic

Add begin, Addend

Topic

Addx after y

Topic

Topic

Topics to be Covered



Topic

Circular Linked List

Topic

Queue using Linked List

Topic

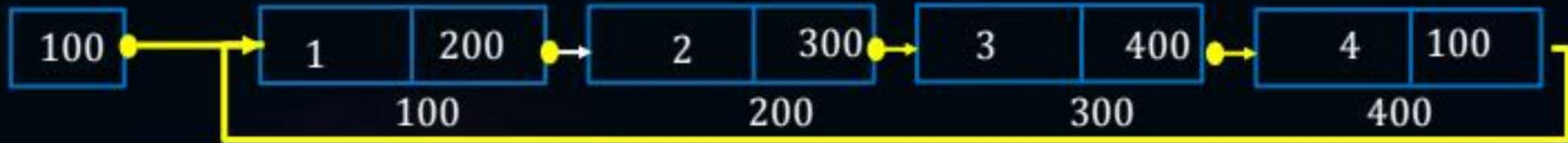
Stack using Linked List

Topic

Topic



Topic : Circular linked List



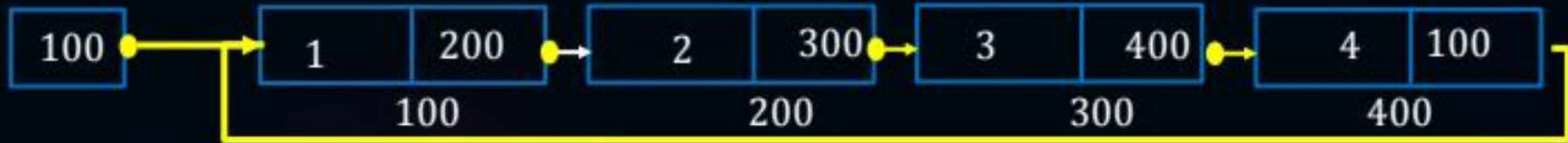
Circular Linked List is a linked list in which

There is no null Terminated pointer. The Last node node of linked list points to first Node of linked list.



Topic : Circular linked List

HP Global variable

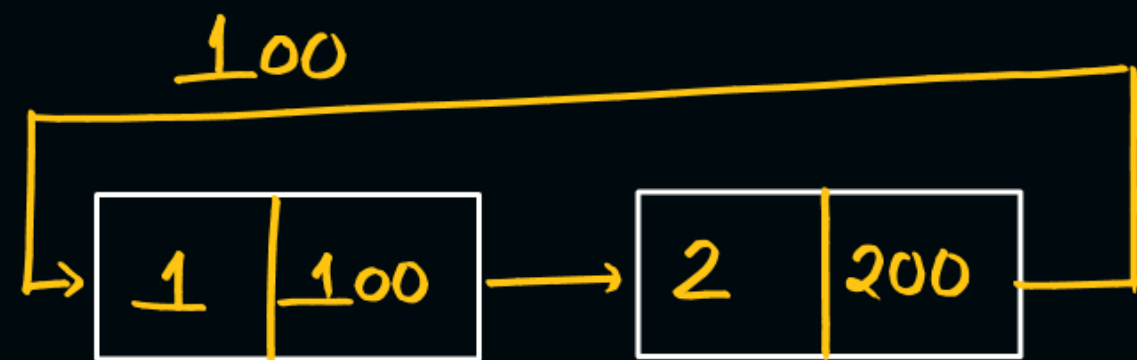


Nonempty

```
void display( node *s) {  
    while (s->next != HP) {  
        printf("%d", s->data);  
        s = s->next;  
        printf("%d", s->data);  
    }  
}
```

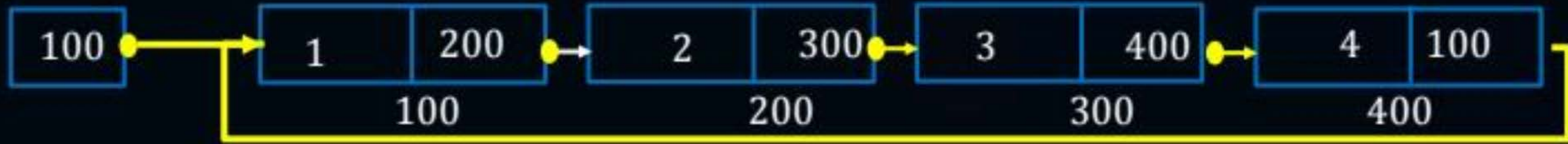
$200 \neq 100$ 1 $S = 200$	$300 \neq 100$ 2 $S = 300$
$400 \neq 100$ 3 $S = 400$	$100 \neq 100$ false


```
void Display(node *s) {  
    if (s) {  
        while (s->next != HP) {  
            printf("%d", s->data);  
            s = s->next;  
        }  
        printf("%d", s->data);  
    }  
}
```





Topic : Circular linked List



```
void traverse(node* s) {  
    if (s == NULL) {  
        printf("List is empty\n");  
        return;  
    }  
    do {  
        printf("%d ", s->data);  
        s = s->next;  
    } while (s != HP);  
}
```

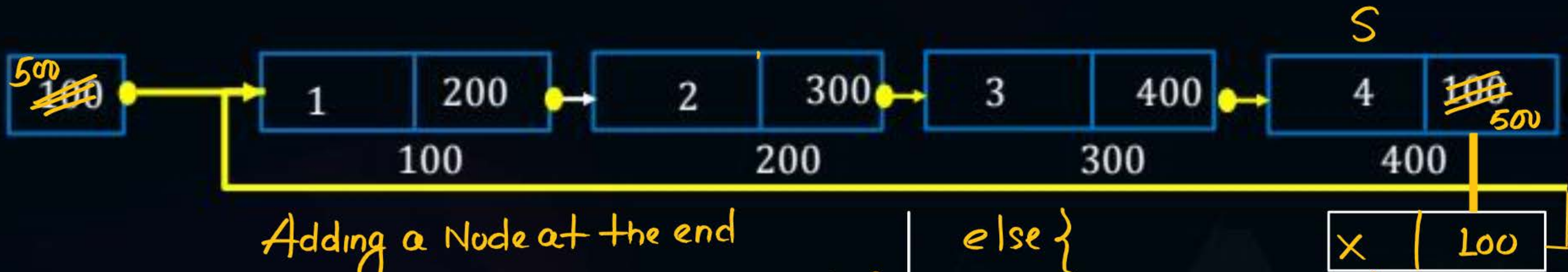
<u>1</u>	<u>2</u>	<u>3</u>	<u>4</u>
$S = 200$	$S = 300$	$S = 400$	$S = 100$
$200 \neq 100$	$300 \neq 100$	$400 \neq 100$	$100 \neq 100$
			<u>false</u>



Adding a Node at the end

```
void Addend (node *s, int x) {
    node *p = getnode(x);
    if (s == NULL) {
        p->next = p;
        HP = p;
    }
}
```

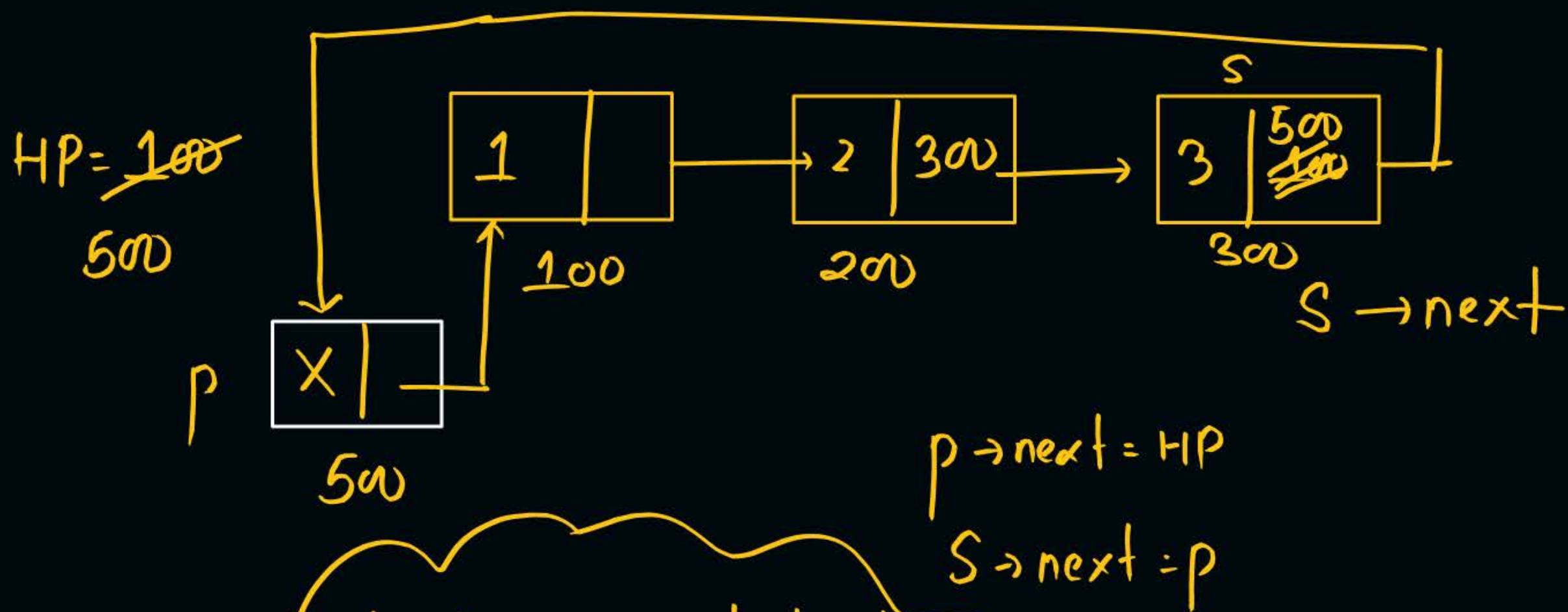
```
else {
    while (s->next != HP)
        S = s->next;
    p->next = HP;
    s->next = p;
}
```



Adding a Node at the end

```
void Addbegin( node *s, int x) {
    node *p = getnode(x);
    if (s == NULL) {
        p->next = p;
        HP = p;
    }
}
```

```
else {
    while (s->next != HP)
        s = s->next;
    p->next = HP;
    s->next = p;
    HP = p;
}
```

Add begin and Addend
both required traversal
& Depends upon length



Let LIST be a datatype for an implementation of linked list defined as follows:

```
typedef struct list {
    int data;
    struct list *next;
} LIST;
```

HW₂

Suppose a program has created linked-lists L1 & L2, whose contents are given in the figure below (code for creating L1 & L2 is not provided here). L1 contains 9 nodes and L2 contains 7 nodes.

Consider the following C-prog segment that modifies the list L1. The number of nodes that will be there in L1 after execution of the code sequence is _____.

```
int find (int query, LIST *list) {
    while (list != NULL) {
        if (list → data == query) return 1;
        list = list → next;
    }
    return 0; }

int main () {
    List ptr1 = L1; ptr2 = L2;
    while (ptr1 → next != NULL) {
        query = ptr1 → next → data;
        if ((find(query, L2))
            ptr1 → next = ptr1 → next → next;
        else
            ptr1 = ptr1 → next;
    }
    return 0;
}
```

Pyq





Question

A circular queue has been implemented using a single linked list where each node consists of a value and a single pointer pointing to the next node. We maintain exactly two external pointers **FRONT** and **REAR** pointing to the front node and the rear node of the queue, respectively. Which of the following statements is/are **CORRECT** for such a circular queue, so that insertion and deletion operation can be performed in $O(1)$ time ?

- I. Next pointer of front node points to the rear node.
- II. Next pointer of rear node points to the front node.

- (A) I only (B) II only
- (C) Both I and II (D) Neither I nor II

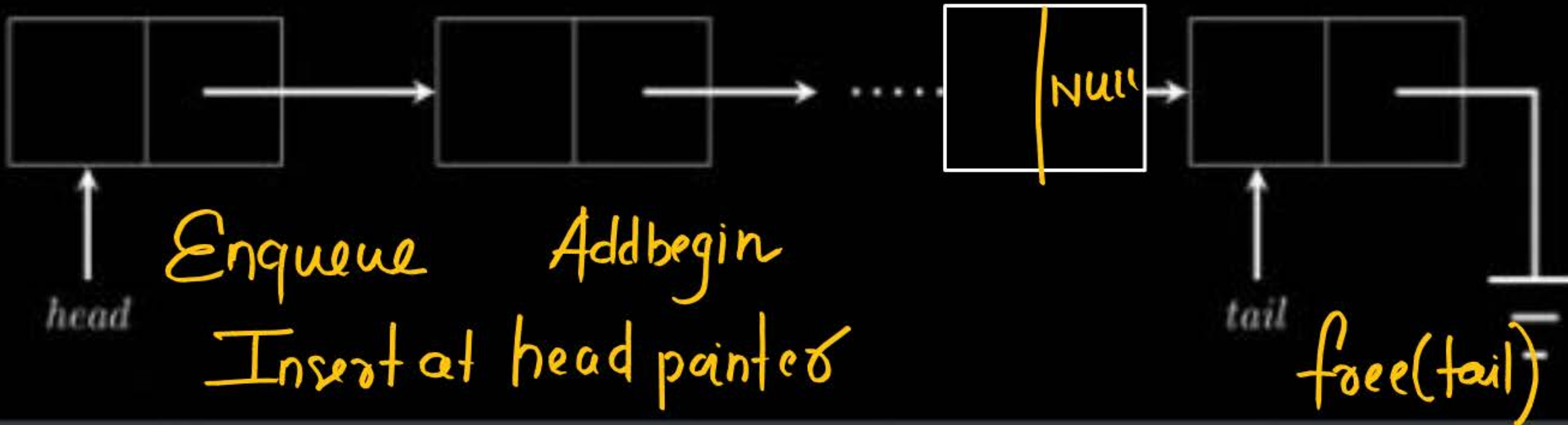
HW 1



Question

$\Theta(1)$

A queue is implemented using a non-circular singly linked list. The queue has a head pointer and a tail pointer, as shown in the figure. Let n denote the number of nodes in the queue. Let 'enqueue' be implemented by inserting a new node at the head, and 'dequeue' be implemented by deletion of a node from the tail.



Which one of the following is the time complexity of the most time-efficient implementation of 'enqueue' and 'dequeue', respectively, for this data structure?

- A. $\Theta(1), \Theta(1)$
- B. $\Theta(1), \Theta(n)$
- C. $\Theta(n), \Theta(1)$
- D. $\Theta(n), \Theta(n)$

dequeue
delete end
traversal
Needed
update tail
Last next to NULL

Stack Implementation using linked list.

push(1)



↑
top

top points to Last element of List

push(x) = Insert at end

push(2) HP=100



100



↑
top

Do we need traversal??

No

p

push(3)



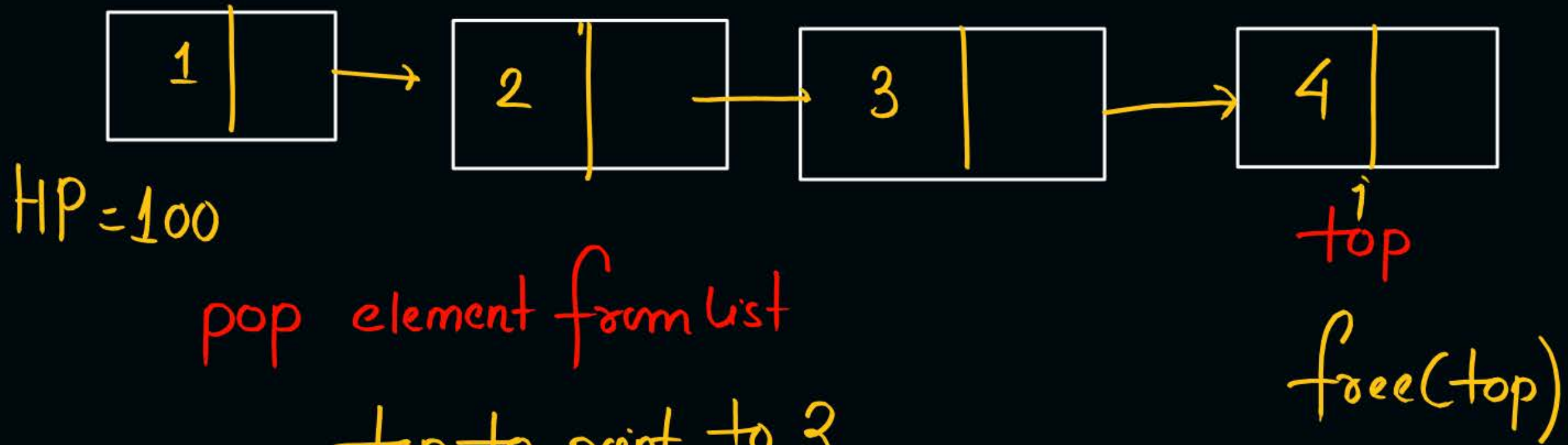
↑
top



HP=100

top → next = p
top = p

Stack Implementation using linked list.



Stack : top points to Last element of list

$$\text{push}(x) = O(1)$$

$$\text{pop}() = O(n) \quad \text{+ traversal needed.}$$

Second strategy

HP = 100
top = 100

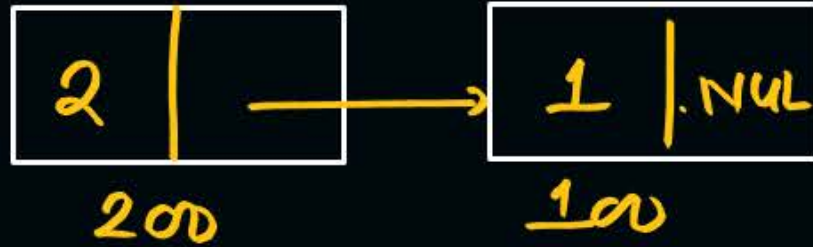


top pointer points to
first element of list

push(2)

HP = 200

top = 200



push required traversal??
No

push = Add begin

push(3)

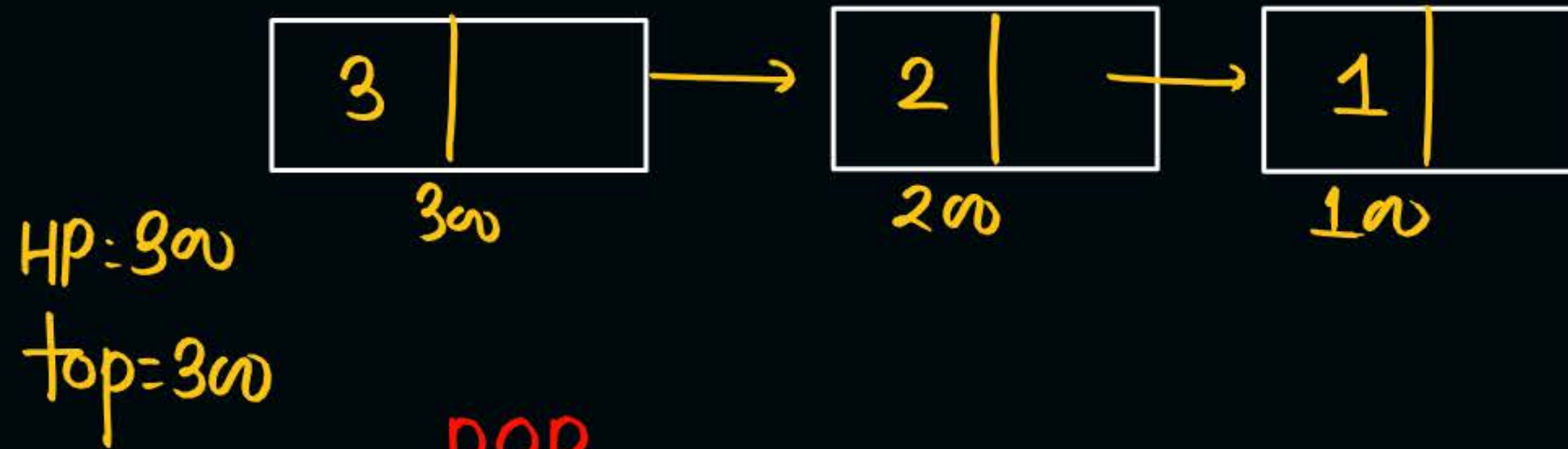
HP = 300

top = 300



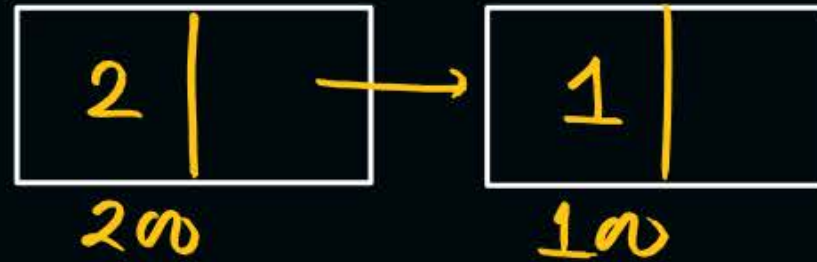
O(1)
Insert at begin.

Second strategy



pop

HP: 200
top: 200



top pointer points to first element of list

Delete Head node

update top and
HP to next element

pop: delete first element

No traversal needed: Hence $O(1)$

THANK - YOU